CampusBridge

Honest student reviews of off-campus housing

HOW IT GETS BUILT
11 steps, 5 phases, in plain English

The whole build, start to finish. Each step is one small thing that works before the next one starts — so at any point there is something real to open in a browser, not a half-finished pile of code.

FOUR WORDS THAT COME UP BELOW

A website is two things working together: something that decides what to show, and somewhere that remembers the information. Everything in the roadmap is one of these four pieces.

The engine
Flask
Sits waiting for visitors, works out which page they asked for, and sends it back.

The page mould
Jinja2 templates
One master page shape that gets filled in with different information each time.

The storage
SQLite
The filing cabinet where properties and reviews are kept so nothing is ever lost.

The address
Deploying
Putting the finished site on a computer that stays online, so it has a public link.

PHASE 1 — SETUP

Steps 1

Get a place to build before building anything.

1 Environment setup

Nothing can be built until the right tools are on the laptop. Python is the language the site is written in. Flask is a small add-on that lets Python run a website. The "isolated workspace" means this project keeps its own copy of everything it needs, so updating something for CampusBridge can never break another project, and the exact same setup can be recreated on any other computer later.

LIKE *Clearing a bench and laying out your own tools before starting a piece of furniture, instead of borrowing from the whole house.*

→ **A working place to build, where the site can already run on my own laptop.**

Python · Flask · Virtual environment

PHASE 2 — BACKEND BASICS

Steps 2–4

Make the site run, look like a real webpage, and remember things.

2 Flask basics

A website is really just a program sitting there waiting. Someone types an address, the program notices, works out which page they asked for, and sends something back. This step builds that listening-and-answering part and nothing else, so at the end the site technically works: open it in a browser and a page appears, even if that page just says "hello".

LIKE *Hiring a receptionist before decorating the office — someone has to pick up the phone before there is anything to talk about.*

→ **Type the address, get a page back — the site is alive, even if it is still blank.**

Routes · Local server

3 Templates

Instead of writing every page by hand, one master layout holds the header, the footer and the general shape of a page, and the individual pages just drop their own content into the middle. Change the master layout once and all eleven pages change with it. This is also what lets the site fill in different information — a property name, a rating — into the same page shape over and over.

LIKE *A form letter: one printed template, with the name and address filled in fresh for each person.*

→ **Every page shares one header, footer and layout, so nothing has to be repeated by hand.**

Jinja2 · HTML · Shared layout

4 Database setup

Reviews have to survive after someone closes the browser, so they cannot live inside the page itself — they need real storage. This step sets up two organised lists: one for properties (name, area, rent, distance from campus) and one for reviews (rating, comment, which property it belongs to). Because each review is tagged with its property, the site can always pull back exactly the right reviews for the page someone is looking at.

LIKE *A filing cabinet with two labelled drawers, where every review slip is filed behind the flat it describes.*

→ **Two organised lists behind the scenes: one for properties, one for reviews, linked together.**

SQLite · Properties · Reviews

PHASE 3 — CONNECTING IT ALL

Steps 5–7

Wire the pages to the stored data so the site actually does something.

5 Browse page

The first page that does something genuinely useful. When a student opens the browse page, the site goes to the storage, asks for every property, and lays them out as a list of cards — name, area, rent, average rating. Nothing here is typed into the page by hand: add a property to the storage and it appears on this page by itself.

LIKE *A noticeboard that refills itself from the filing cabinet every time someone walks past it.*

→ **A real browse page: every property near campus, with its rent, area and rating at a glance.**

Read from database · List view

6 Property detail page

Clicking a property on the browse page opens a page about only that property: its details at the top, and underneath it every review students have written about it, newest first. Each property gets its own web address, so a student can send a friend a link to one specific flat rather than to the whole list.

LIKE *Pulling one file out of the cabinet and reading everything filed behind it.*

→ **One page per property, with its details and every review students have left on it.**

Links · Ratings · Review list

7 Write a review

Up to now information only flowed one way, out of storage and onto the screen. This step opens the other direction: a student fills in a rating and a comment, presses submit, the site checks the important fields are not empty, files the review behind the right property, and sends them back to the page where their review is now visible. This is the moment CampusBridge becomes a community site rather than a list.

LIKE *Adding the suggestion box to the wall, and actually reading what goes into it.*

→ **A student writes a review, hits submit, and it shows up on the property page straight away.**

Form · Saving data · Basic checks

PHASE 4 — CONTENT & POLISH

Steps 8–9

Fill it with real reviews and make it look like CampusBridge.

8 Seed real content

A review site with nothing in it is useless, so real properties near the university and real reviews gathered from students are loaded in before anyone visits. This also stress-tests the site with messy, real-world content — long comments, unusual names, missing information — which is very different from the tidy test data used while building.

LIKE *Stocking the shelves before opening day, rather than asking the first customer to bring their own goods.*

→ **Real listings and real student reviews, so the site is genuinely useful the moment it opens.**

Research · Real reviews

9 Styling

Everything works at this point, but it looks like a document rather than a website. Here the navy, teal and coral brand colours go on, along with the typefaces, spacing, cards and buttons — and the layout is made to work on a phone, since most students will open it on one. Looks are not decoration on a review site: it has to feel trustworthy enough that a student is willing to write something honest into it.

LIKE *Painting and furnishing a building that is already structurally sound.*

→ **It stops looking like a school project and starts looking like a product students would trust.**

Brand colours · Typography · Works on phones

PHASE 5 — SHIP IT

Steps 10–11

Check it as a real user would, then put it in front of students.

10 Testing

Every path gets clicked through the way a real visitor would move: browse, open a property, read reviews, write one, go back, try it on a phone. Then the awkward cases on purpose — submitting an empty form, a very long comment, a property with no reviews yet, a link to something that does not exist. Anything confusing or broken gets fixed now, while it is cheap, rather than in front of a real user.

LIKE *Test-driving the car around the block before handing over the keys.*

→ **Every path clicked through at least once, with the awkward bits found and fixed first.**

Click-through · Odd inputs · Fixes

11 Deploy

Until now the site only exists on one laptop. Deploying copies it onto a computer that stays online all the time, so it has a proper web address anyone can open from their own phone with nothing to install and no login. The laptop can be closed and the site stays up — which is the difference between a project and something students can actually use.

LIKE *Moving out of the workshop and into a shop with a street address and opening hours.*

→ **A public web address anyone can open on their phone — no install, no login needed.**

Hosting · Public link

ONCE IT IS LIVE, ONE REVIEW TRAVELS LIKE THIS

Student writes it — Fills in the review form on a property page
→
Site checks it — Makes sure nothing important is missing
→
Storage keeps it — Saved alongside that property for good
→
Everyone reads it — Appears on the property page for the next student

Built from scratch using Python, Flask, SQLite and Jinja2 — no-code tools were intentionally avoided to build real technical skills.